

Rapport – Quiz i Java med JavaFX

Namn: Herman Bengtsson

Inledning

I detta projekt utvecklade vi en quiz-app i Java som låter användaren att skapa konto, logga in, välja mellan olika quizkategorier och starta ett quiz inom den valda kategorin. Användarens resultat sparas för att senare kunna följa sin prestation över tid.

För användargränssnittet (GUI) valde vi JavaFX framförallt då man lätt separerar layout och logik. Frågor och kategorier lagras i JSON-filer, vilket gör det smidigt att ändra eller lägga till innehåll utan att behöva ändra koden/logiken. Användarinformation inklusive lösenord och resultat sparas också i JSON, men med säker kryptering för lösenordet via BCrypt för att skydda användardata.

Projektet genomfördes enligt Scrum-ramverket, med regelbundna sprintar, dagliga stand-ups, planeringsmöten, sprint reviews och retrospektive. Detta gav oss en strukturerad arbetsprocess och en tydlig gemensam målbild.

Sammanfattning

Resultatet blev en fullt fungerande quiz-app med användarhantering, val av olika kategorier och lagring av resultat. Applikationen fungerar enligt våra krav och är användarvänlig.

Samarbetet i gruppen fungerade mycket bra. Även om kunskapsnivån varierade har alla bidragit efter bästa förmåga, och gruppen har varit hjälpsam och stöttande för att se till att alla förstod och kunde delta aktivt i utvecklingen.

Scrum-strukturen var inledningsvis utmanande eftersom ingen av oss hade arbetat med ramverket tidigare. Till en början kändes det som att det bromsade arbetet mer än det hjälpte. Men efter vi blev mer bekväma med ramverket började processen flyta på bättre, och vi fick en tydligare överblick över vad som var klart, vad som pågick och vad som återstod att göra. Detta gav oss en gemensam förståelse av projektets mål och underlättade planeringen betydligt.

En av de största svårigheterna med att arbeta i en grupp kring ett mindre projekt är att det är svårt att skapa helt isolerade uppgifter. Trots planering uppstod situationer där olika delar av koden var beroende av varandra, vilket ibland ledde till att man arbetade utanför det som var planerat. Detta kunde i sin tur skapa merge-konflikter i GitHub som behövde lösas innan merge. Trots dessa utmaningar gav Scrum-ramverket oss en tydlig helhetsbild av projektet. Jag upplever att ramverket är särskilt effektivt i större projekt, där arbetsuppgifter kan delas upp mer isolerat och där beroenden mellan olika delar av koden är tydligare definierade. Men det har ändå gett oss värdefull erfarenhet och struktur även i detta mindre projekt.

Eftersom jag har lite mer erfarenhet av programmering än resten av gruppen var det en utmaning att hitta balansen mellan att bidra mycket till projektet och samtidigt säkerställa att resten av gruppen hängde med. Jag försökte undvika att ta över för mycket av arbetet, eftersom det riskerar att göra koden svårare för andra att förstå och fortsätta på. Jag försökte därför anpassa mitt arbete, förklara lösningar och hjälpa till när det behövdes, så att alla kände att dem kunde bidra. Jag såg också till att kommentera koden tydligt för att underlätta för hela gruppen. Processen blev även nyttig för mig, eftersom jag verkligen behövde sätta mig in i koden för att kunna förklara den på ett tydligt sätt. Det var roligt eftersom jag fick testa mina kunskaper från ett annat perspektiv, vilket också gjorde att jag lärde mig mer och man blir tydligt medveten om vilka delar man behöver förbättra.

I slutet av projektet satt vi alla tillsammans i gruppen i "Code with Me" och skrev klart de sista delarna, fixade buggar och implementerade nya funktioner. Det var mycket givande eftersom man får allas input och idéer i realtid när man kodar, vilket ger olika alternativ för hur saker kan lösas. Det var särskilt värdefullt då man kunde få saker förklarade direkt eller förklara för andra i samma stund. Det borde vi ha gjort tidigare, eftersom det hjälper hela gruppen att få en bättre helhetsbild av vad koden gör och varför den är skriven som den är. Dessutom hjälper det till att undvika många mergekonflikter, som lätt uppstår i början av mindre projekt när allas delar är beroende av varandra.

Designval

Fokus låg på att skapa en flexibel kod för att enkelt utveckla den samt en tydlig ansvarsfördelning där OOP styr designen.

Motivering av designval: OOP och klassstruktur

Applikationen består av tydliga objekt såsom Quiz, Question, Category, User och Result, vilket gjorde OOP till ett naturligt val. Genom att kapsla in data i egna klasser kunde varje del av applikationen få ett specifikt ansvar. Till exempel:

Quiz hanterar quizlogiken, poängräkning, frågeindexering och svarskontroll.

Question representerar en enskild fråga med frågetext, svarsalternativ och korrekt index.

User innehåller användarinformation och resultatlista.

LoginService ansvarar för autentisering och lösenordshantering genom säkra hashade lösenord (BCrypt).

Denna struktur gjorde koden enkel att vidareutveckla utan att riskera konflikter mellan logik och datahantering. Dessutom minskade risken att framtida funktioner t.ex fler frågor/kategorier eller annan lagring skulle kräva att mycket av koden behöver skrivas om.

Datahantering och val av JSON + Jackson

Frågor och kategorier lagras i JSON-filer, eftersom formatet är både lättläst och flexibelt. Vid tillägg eller ändring av en fråga krävs ingen omskrivning av koden/logiken. Jackson-biblioteket användes eftersom det kan serialisera och deserialisera Java-objekt utan manuell filhantering. Klassen JSONFileReader ansvarar för att läsa in frågekategorier, vilket helt separerar filhantering från quiz-logiken.

Användardata sparas också i JSON. UserRepository hanterar läsning och skrivning av Users.json och använder Singleton-mönstret för att säkerställa att alla controllers arbetar mot samma lista med användare för att säkerställa att ingen data "försvinner".

JavaFX, MVC-inspirerad controllerstruktur

Vi valde JavaFX framför Swing eftersom JavaFX:

Tydlig separation mellan UI och logik via FXML.

Modernare komponenter och bättre styling med CSS om man skulle vilja lägga till det.

Vi försökte följa en MVC struktur så mycket som möjligt där logiken ligger i mainProgram-paketet och UI hanteras i controllers under com.example.quizFX. Exempel:

QuizController hanterar timer med Timeline, kontrollerar svar och sparar resultat.

ResultsController renderar en TableView som dynamiskt visar användarens sparade resultat.

Genom att placera all logik utanför controllers undviks "spagettikod" där UI styr programmets funktionalitet. Själva logiken kan därför utvecklas och ändras utan att påverka utseendet. Men det kan självklart förbättras och delas upp ännu mer.

Säkerhet och lösenordshantering

Eftersom applikationen hanterar användare med lösenord användes BCrypt för säker

lösenordslagring. Istället för att lagra lösenord direkt i JSON-filen lagras endast hash-värden, vilket hanteras i LoginService.

Sammanfattning:

Projektets struktur baseras på tydlig ansvarsfördelning, autentisering och lättutbyggbar filbaserad datalagring. En kombination av OOP, JSON, JavaFX och en någorlunda MVC separation mellan logik och UI/utseende har lett till en lösning som är:

Modulär och lätt att underhålla

säker gällande lösenordshantering

skalbar för utbyggnad av fler frågetyper, användarroller eller databaslagring i framtiden

Hantering av bibliotek och byggverktyg

För att förenkla hanteringen av externa bibliotek som JavaFX och Jackson har vi använt Maven som byggverktyg. Vi har definierat deras beroenden i en pom.xml fil för att alla som arbetar med projektet automatiskt får rätt versioner av biblioteken när de kör projektet. Detta gör att det blir enkelt att starta upp projektet på olika datorer utan manuella konfigurationer.

Denna lösning minskar risken för kompatibilitetsproblem och gör projektet enklare att köra på olika system/datorer.

////////////////////////////////////

Reflektioner kring Scrum och sprintar

Sprint Planning

Under våra sprint planning möten märkte vi tidigt att våra beskrivningar av issues ofta var ottydliga och ibland alldeles för generella. Många uppgifter var ganska stora och omfattande, vilket gjorde det svårt att få en tydlig bild av exakt vad som behövde göras. Detta ledde till att det ibland blev oklart när en uppgift egentligen var färdig, och det blev svårare att dela upp arbetet mellan gruppmedlemmarna på ett effektivt sätt.

Vi insåg också att när issues är stora och ospecifika är det svårare att gå tillbaka i efterhand och se vilka delar som faktiskt blivit implementerade, vilket försvårar både uppföljning och felsökning. Det gjorde att vi ibland fick lägga mer tid på att förklara och diskutera vad som ingick i varje issue, vilket tog tid från själva utvecklingen.

Efterhand, när vi fick mer rutin på sprint planning, blev vi bättre på att bryta ner stora uppgifter i mindre, mer hanterbara delar. Det hjälpte oss att skapa tydligare och mer konkreta beskrivningar, vilket gjorde det enklare att planera, fördela och genomföra arbetet. Med

dess mindre delar blev det också lättare att följa upp och se hur mycket som var gjort under sprinten. En lösning vi hade på det var att skapa sub-issues av redan existerande issues vilket gav en tydlig bild om vad som skulle göras.

Sammanfattningsvis kan vi säga att vår sprint planning blev mer strukturerad och effektiv med tiden, mycket tack vare att vi lärde oss vikten av att formulera detaljerade och konkreta issues och att dela upp stora uppgifter i mindre delar.

Sprintar

Jag tycker att våra sprints överlag gick bra. Vi hade dagliga Scrum möten med god närvaro. Trots att vi hade sprint-planning möten behövde vi ofta ha mindre planeringsmöten under veckan eftersom allt inte var helt glasklart från början. Till exempel hade vi ganska vaga beskrivningar av vad vi ville uppnå, och alla steg på vägen mot målet var inte dokumenterade. Vi gick ofta igenom detaljerna muntligt på våra möten, men eftersom vi inte skrev ner allt tydligt kunde man tappa bort sig lite i mitten av sprinten. När det hände tog vi därför ett extra möte för att säkerställa att alla var med på vad som skulle göras. Det jag tycker att vi gjorde bra var att vi var flexibla och kunde stanna upp mitt i sprinten för att rätta till det som behövde förbättras. Något vi definitivt kunde göra bättre är att beskriva målen tydligare och dokumentera alla steg på vägen dit.

Styrka: Flexibilitet i gruppen

Förbättring: Tydligare dokumentation och målformulering

Sprint Review

Till en början hade vi ingen struktur i våra sprint reviews och hade inget att direkt gå efter, utan vi hade endast en diskussion om vad vi hade gjort under sprinten. Till 3:e sprinten började vi följa en mall vilket hjälpte oss mycket, och vi missade inga viktiga punkter. Mallen kändes dock överflödigt och onödigt stor för vårt projekt, så om vi hade haft mer tid så hade man kunnat skriva en egen mall som är mer anpassad för vår grupp så att vi optimerar våra möten.

När vi kände oss mer bekväma med strukturen och mallarna kändes det som att mötena var givande. Vi igenom allt arbete som var färdigt och demonstrerade det för gruppen. Vi körde programmet och delade skärmen så att alla kunde se exakt vad som hade gjorts. Detta hjälpte oss att upptäcka småfel och möjliga förbättringar som vi kanske annars hade missat. Vi gick också igenom de mål vi hade satt upp för sprinten och checkade av hur väl vi hade uppnått dem.

En sak vi hade kunnat göra bättre är att förbereda våra presentationer mer och lägga upp

dem som om vi skulle visa programmet för någon som aldrig sett det tidigare. Eftersom det bara var gruppen som deltog i mötet blev det ofta att vi tog för givet att alla redan visste vad som var gjort, vilket ibland gjorde att viktiga detaljer utelämnades. Genom att tänka sig inte mer i en "verklig" miljö hade våra reviews kunnat bli tydligare och mer givande.

////////////////////////////////////

Sprint Retrospective

Jag tycker att våra retrospective möten har varit mycket bra och givande under hela projektet. Alla i gruppen har bidragit med konstruktiv kritik på ett bra sätt. Det har gjort att mötena känts relevanta och meningsfulla, samtidigt som de hjälpt oss att utvecklas både som grupp och i vårt arbete med projektet.

En stor förbättring vi gjort med tiden är att vi började följa en tydlig mall för våra möten. Denna mall hjälpte oss att hålla fokus på viktiga punkter, så att vi inte glömde något viktigt. Tack vare mallen blev våra anteckningar också mycket tydligare och mer strukturerade. Det gjorde det enkelt för oss att senare gå tillbaka och läsa igenom vad vi kom fram till på varje möte, vilket undviker att man gör om samma misstag. En negativ sak med mallen vi upptäckte var att den kunde kännas väldigt överflödigt och tog upp massa saker som vi inte hade nytta av. Och att det kändes som att man repeterade sig mycket, en lösning på det hade kunnat vara att man tar del av olika mallar och sedan skapar en egen mall som är anpassad för gruppen. Det har vi tyvärr inte haft tid med, men om vi skulle fortsatt med arbetet hade det varit ett bra alternativ.

Samarbetet under dessa möten har varit väldigt bra i. Vi har alla känt oss trygga med att dela våra tankar och förslag, vilket har skapat en öppen dialog där alla har sagt vad dem tycker.

Trots detta finns det fortfarande saker vi kan bli bättre på. En viktig sak är att koppla kritiken och anteckningarna tydligare till konkreta exempel från vårt arbete. När man har tydliga exempel blir det lättare att förstå exakt vad som menas och vad vi behöver förändra. Dessutom skulle vi kunna göra våra anteckningar mer utförliga. Det innebär att vi skriver ner mer detaljer så att man i framtiden kan läsa dem och direkt förstå vad mötet handlade om och vilka åtgärder vi beslutat.

Ett exempel från ett av våra möten var en förbättringspunkt: "Tydligare och mer beskrivande mål". Den kan utvecklas så att den förklarar mer i detalj vad som saknades och hur vi kan förbättra oss, så att anteckningarna kan fungera som vägledning när vi senare vill undvika att göra samma misstag igen.

Sammanfattningsvis har våra retrospektive möten varit mycket givande och roliga. Genom bra samarbete och en tydlig mall har vi kunnat utvecklas mycket som grupp och gjort vårt projekt bättre.

////////////////////////////////////

Reflektion kring rollen som Scrum Master

När jag var Scrum Master låg mitt huvudfokus på att se till att ingen i teamet hade större hinder i sitt arbete och att alla hade en tydlig bild av sprintmålet. Jag såg också till att alla visste vilka uppgifter eller issues de ansvarade för. Jag försökte alltid vara extra tillgänglig, bland annat för att hjälpa till med tekniska problem som uppstod, till exempel relaterade till Git. Jag upplevde rollen som väldigt givande och inte särskilt svår, mycket tack vare att alla i gruppen är både hjälpsamma och drivna. Det gjorde arbetet betydligt enklare.

Vi har dessutom haft en väldigt öppen kommunikation i teamet, vilket underlättade. Till exempel, om någon haft sjukt barn eller blivit sjuk själv och därför inte hunnit färdigställa sina uppgifter, har vi kunnat justera planeringen utan problem. Det jag framför allt lärde mig som Scrum Master var hur viktigt det är med kommunikation och att fånga upp problem tidigt innan de hinner påverka sprinten för mycket. Jag insåg också hur mycket skillnad det gör när alla i gruppen vågar vara öppna med hur det går och säga till när något inte funkar eller när man behöver hjälp.

Som Scrum Master handlar det inte om att bestämma, utan mer att stötta och underlätta så att alla kan göra sitt bästa. Jag märkte också hur viktigt det är att vara flexibel, även om man har en bra planering så kan saker ändras snabbt, till exempel om någon blir sjuk eller något oväntat händer. Sammanfattningsvis lärde jag mig mycket om hur viktigt det är med bra kommunikation, tydlighet och ett öppet arbetsklimat för att ett team ska fungera bra tillsammans.

